

SIMATS ENGINEERING

CO3 ASSESSMENT TOOL 1

NAME: JAIVANT.S

REG NO: 192311326

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Q13. Analyze the Impact of Varying Recursion Limits on Binary Search

Introduction

Binary Search is one of the most efficient searching algorithms used to find an element in a **sorted array or list**. It works by repeatedly dividing the search space into two halves until the target element is found. Binary Search can be implemented using **recursive** or **iterative** methods. In the recursive method, each function call is stored in the system's call stack, and the maximum number of recursive calls is controlled by the **recursion limit**. Understanding the effect of recursion limits is important, especially in memory-constrained environments.

Objectives

- Study the effect of recursion limits on Binary Search.
 - Compare recursive and iterative Binary Search.
 - Analyze execution time and stack memory usage.
 - Understand how recursion depth affects system stability.
 - Justify why iterative Binary Search is preferred in many applications.
-

Requirements

- Sorted input data.
 - Binary Search algorithm (recursive or iterative).
 - Python programming language.
 - Execution time measurement.
 - Observation of stack memory usage.
-

Working of Recursive Binary Search

1. Find the middle element of the sorted list.
 2. Compare it with the target element.
 3. If both are equal, return the result.
 4. If the target is smaller, recursively search the left half.
 5. If the target is larger, recursively search the right half.
 6. Continue until the element is found or the search space becomes empty.
-

Impact of Recursion Limits

The recursion limit determines the maximum number of recursive function calls allowed by the programming language.

- A **low recursion limit** may terminate the program with a recursion error if the limit is exceeded.
- A **higher recursion limit** allows deeper recursive calls but increases stack memory usage.

- Binary Search has a recursion depth of $\log_2(n)$, so it rarely reaches the recursion limit for normal datasets.
 - Function call overhead in recursion slightly increases execution time compared to the iterative method.
 - Increasing the recursion limit does not improve the algorithm's time complexity.
-

Execution Time Analysis

- Recursive Binary Search requires additional time for every function call.
 - Iterative Binary Search executes inside a loop and avoids function call overhead.
 - Both approaches perform the same number of comparisons.
 - For small datasets, the difference in execution time is negligible.
 - For large datasets, the iterative version is generally faster.
-

Stack Usage Analysis

- Recursive implementation uses the system call stack.
 - Every recursive call creates a new stack frame.
 - Stack usage increases with recursion depth.
 - Iterative implementation uses constant memory because no recursive calls are made.
 - Therefore, iterative Binary Search is more memory-efficient.
-

Effect on System Stability

- Low recursion limits may stop program execution unexpectedly.
 - Increasing recursion limits improves stability only up to a certain level.
 - Very high recursion limits may increase memory consumption and reduce system efficiency.
 - Recursive implementations are more likely to experience stack overflow in algorithms with deep recursion.
 - Binary Search is relatively safe because its recursion depth grows logarithmically.
-

Comparison of Recursive and Iterative Binary Search

- Recursive Binary Search is easier to understand and write.
 - Iterative Binary Search uses less memory.
 - Recursive implementation has function call overhead.
 - Iterative implementation is slightly faster.
 - Recursive method depends on recursion limits.
 - Iterative method avoids recursion errors and stack overflow.
-

Advantages of Recursive Binary Search

- Simple and readable code.
 - Easy to understand Divide and Conquer approach.
 - Suitable for educational purposes.
 - Efficient for moderate-sized datasets.
-

Limitations of Recursive Binary Search

- Uses additional stack memory.
 - Depends on recursion limits.
 - Slightly slower due to recursive function calls.
 - Not suitable for memory-constrained systems.
-

Why Iterative Binary Search is Preferred

The iterative approach is preferred because:

- It uses **O(1)** auxiliary space.
 - It does not depend on recursion limits.
 - It avoids stack overflow.
 - It has lower execution overhead.
 - It is more suitable for embedded systems and real-time applications.
 - It provides better overall system stability.
-

Time Complexity

- **Best Case:** $O(1)$
 - **Average Case:** $O(\log n)$
 - **Worst Case:** $O(\log n)$
 - **Space Complexity (Recursive):** $O(\log n)$
 - **Space Complexity (Iterative):** $O(1)$
-

Program:

```
1 def binary_search(arr, key):
2     low, high = 0, len(arr) - 1
3
4     while low <= high:
5         mid = (low + high) // 2
6
7         if arr[mid] == key:
8             return mid
9         elif arr[mid] < key:
10            low = mid + 1
11        else:
12            high = mid - 1
13
14    return -1
15
16
17 arr = [10, 20, 30, 40, 50, 60, 70]
18 key = int(input("Enter element: "))
19
20 result = binary_search(arr, key)
21
22 if result != -1:
23     print("Element found at index", result)
24 else:
25     print("Element not found")
```

Output:

```
Enter element: 50  
Element found at index 4  
  
=== Code Execution Successful ===
```

Conclusion

Binary Search is an efficient searching algorithm based on the **Divide and Conquer** technique. Although the recursive implementation is simple and easy to understand, it depends on the recursion limit and consumes additional stack memory. The iterative implementation eliminates these issues by using constant memory and avoiding recursive function calls. In constrained environments where memory and stability are important, the **iterative approach is the preferred choice** because it offers better performance, lower memory usage, and improved reliability.
